DATA 260 Homework 9

Name: Akanksha Shukla

GitHub repository: https://github.com/akankshashuklaWork/data260-7838

Personal configuration

Value	Result	How I calculated it
SID4	7838	Last four digits of my SJSU Student ID
PORT_BASE	8638	8000 + (7838 mod 900) = 8000 + 638
PREFIX	s7838	"s" + SID4 (every Redis key I create starts with it)
SEED	7838	SID4
VERIFY_SEED	267838	260000 + SID4
DOMAIN_ID	6	7838 mod 8 = 6, which is Rental Housing Listings
Hardware	MacBook Pro	Apple M4 Pro chip, 48 GB memory
Local model	qwen2:7b	I run it locally through Ollama (embeddings: all-MiniLM-L6-v2)
Tagged commit hash	TO FILL IN	Run git rev-parse hw9^{} after tagging hw9, then paste the hash here

What this homework is about

- In HW8 I built a system where a question travels through three agents (Planner, Writer, Reviewer) over Kafka. It works, but every question takes several seconds because a language model runs each time.
- In HW9 I add **caching** in front of that system. A cache remembers answers I have already produced, so a repeated question can be answered almost instantly.
- But caching can also give a **wrong answer fast**. So the real goal of this homework is to measure the trade-off between speed and correctness, not just to make things faster.
- **Part 1:** I count views of a listing in Redis. **Part 2:** I build a semantic cache that recognizes questions with the same meaning. **Part 3:** I measure when that cache is wrong and what it costs. **Part 4:** I build two agents that check and improve each other's answers.

Concepts I use in this report

- **Cache:** a fast place where I keep an answer I already worked out, so I do not have to work it out again.
- **Redis:** a very fast in-memory data store. I use it for the view counter and for the cache.
- **INCR:** a Redis command that adds 1 to a number in one safe step, even if many programs call it at the same time.
- **Cache hit / miss:** a hit means I found the answer in the cache. A miss means I did not, so the real work has to run.
- **Embedding:** a list of numbers (384 in my case) that stands for the meaning of a sentence. Sentences with similar meaning get similar numbers.
- **Cosine similarity:** a score from 0 to 1 that says how close two embeddings are. 1.0 means the same meaning.

- **Vector index:** a Redis feature that quickly finds the stored embedding closest to a new one.
- **Threshold:** the minimum similarity score I require before I call something a hit.
- **False hit:** a cache hit that returns the wrong answer because the two questions only look alike.
- **Stale answer:** a cached answer that was correct when I stored it, but the data behind it has changed since.
- **Lesson:** a saved note about a mistake that keeps repeating, plus how to avoid it next time.

Part 1 - Redis view counter

What I wanted to do

- My primary entity is a **rental housing listing**. Every time someone opens a listing, its view counter should go up by one.
- Updating the database on every single view would be slow. So I count the views in Redis, which is very fast, and write the number to the database only once in a while.

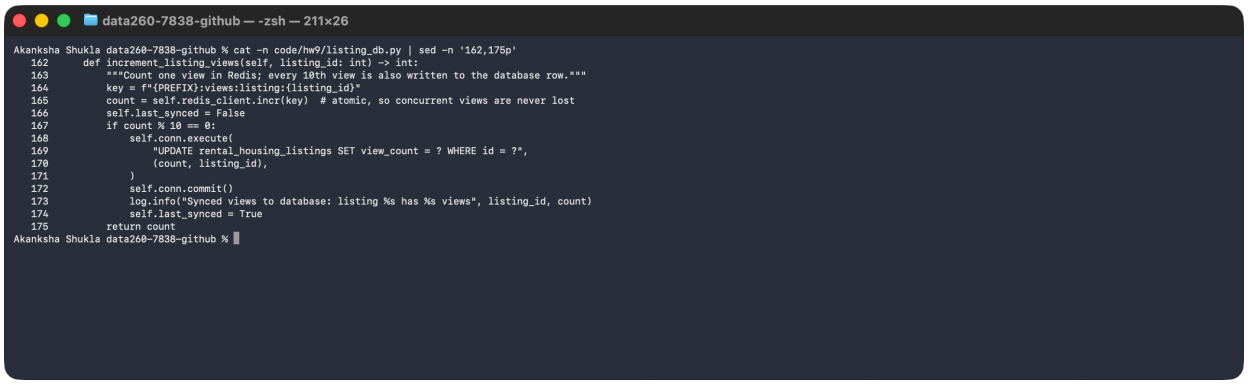
What I built

- I made a class **ListingDatabase** (code/hw9/listing_db.py). It connects to a SQLite database file named s7838_rel.db and to Redis.
- The database has a table called rental_housing_listings with six sample listings (for example Maple Court). Each row has a view_count column.
- I added **one new method**: increment_listing_views(listing_id). It is the only new method this part asks for.
- The Redis key is **s7838:views:listing:{id}**. It starts with my PREFIX, as the assignment requires.

How my method works, step by step

- **Step 1:** I call Redis INCR on the key. Redis adds 1 and gives me back the new count. INCR is safe even if many users view at once, so no view is lost.
- **Step 2:** I check count % 10 == 0 (is the count a multiple of 10?). If yes, I update that listing's row in the database with the current count and log 'Synced views to database'.
- **Step 3:** I return the current count.
- **Trade-off I accepted:** if the program crashed after view 17, the database would still say 10, but Redis would still hold 17. That is fine for a view counter, and it saves a database write on 9 out of every 10 views.

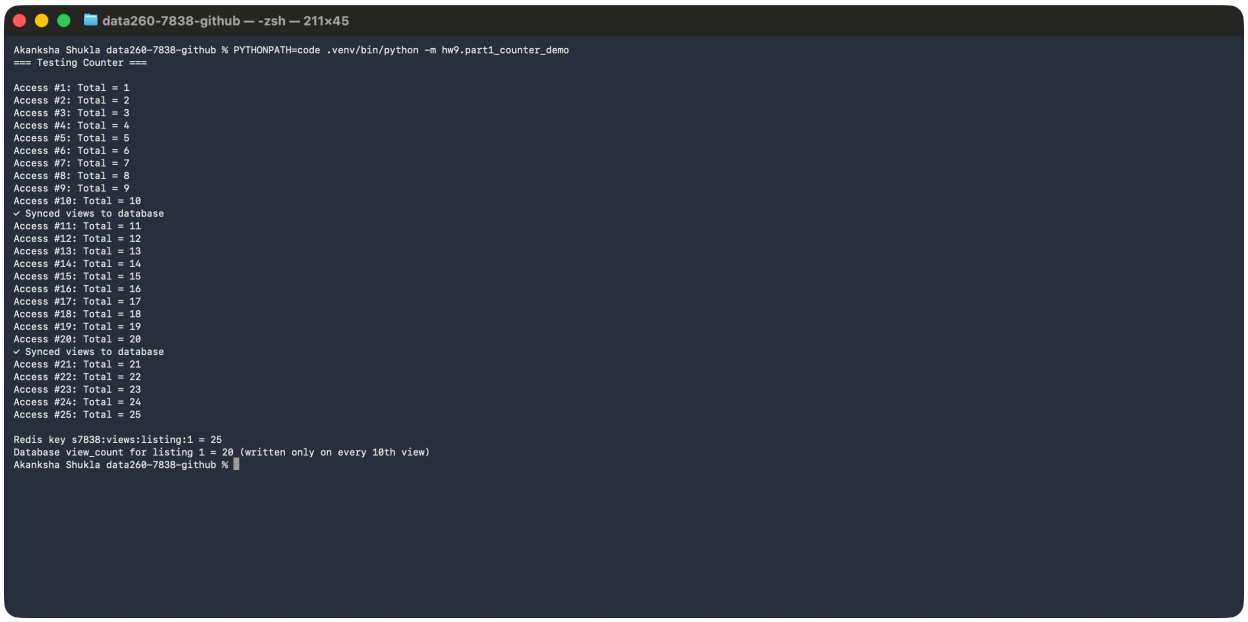
Screenshot - my increment_listing_views code



What I saw when I ran it

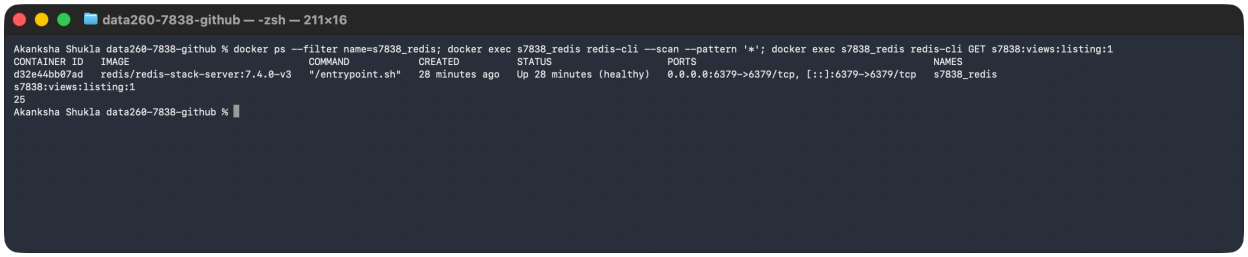
- I ran 25 accesses. The output matches the expected output in the assignment: Access #1 to #25 with the totals, and the check-mark line after #10 and #20.
- My demo prints the check mark when the method reports that it synced. The method itself writes the log message.

Screenshot - 25 accesses (output matches the expected output)



- At the end, Redis holds **25** but the database holds **20**. That proves the database was only written on the 10th and 20th view.
- The second screenshot shows the Redis container running and the only key it holds, which starts with s7838.

Screenshot - the Redis container and the counter key



Part 2 - Semantic response cache

What I wanted to do

- A normal cache only helps when the exact same text is asked again. People ask the same thing in different words, for example 'What is a security deposit used for?' and 'What is a security deposit meant to cover?'.
- A **semantic** cache compares meaning, not text. So the second question can reuse the answer to the first.

Where my cache sits

- I put it at the **ingress**, which means the front door of my Q&A endpoint (POST /ask on my port 8638), in front of the HW8 agent pipeline.

- **When there is a miss:** (1) I embed the question, (2) I search the Redis vector index and find nothing close enough, (3) I send the question through the normal HW8 path: Kafka, then Planner, Writer and Reviewer using qwen2:7b, (4) I store the result, (5) I return the answer with is_cached=false.
- **When there is a hit:** I return the stored final answer straight away without running any agent, with is_cached=true and the similarity score.

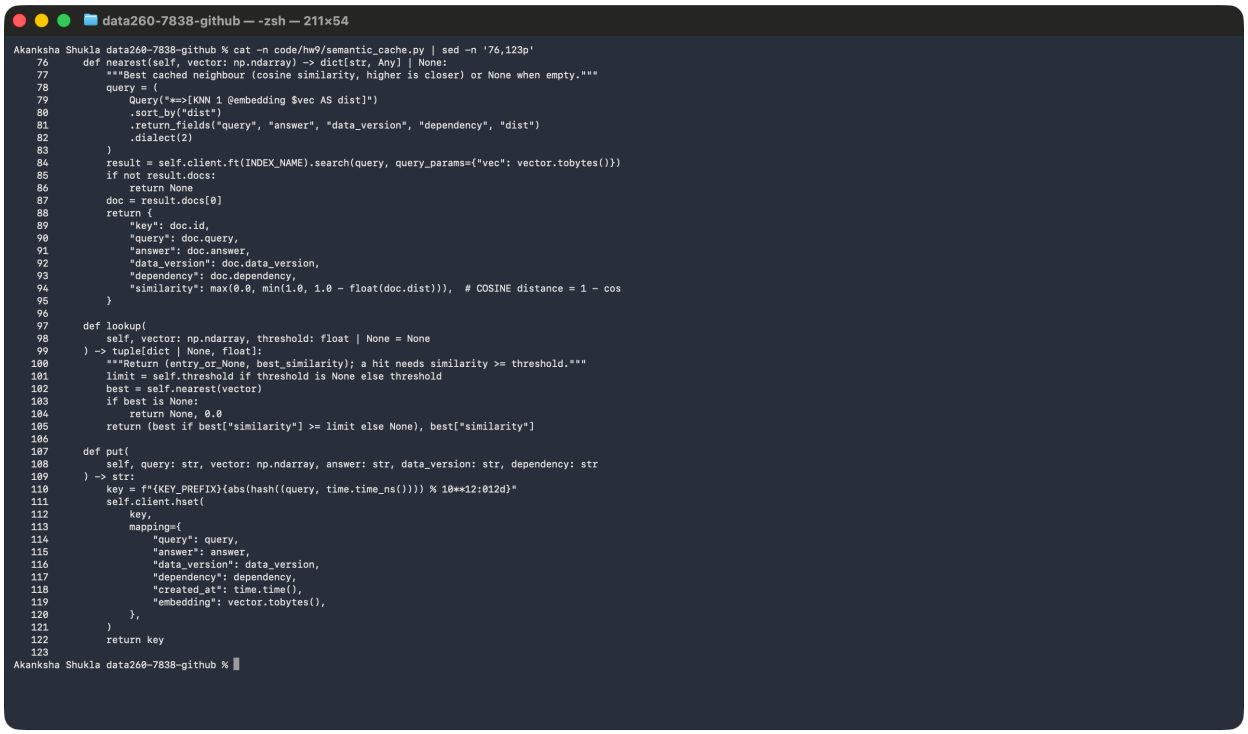
What I store for each answered question

- The **query** text.
- Its **embedding** (made with SentenceTransformers all-MiniLM-L6-v2) as bytes, in a Redis vector index that uses cosine distance.
- The **final answer** from the pipeline.
- A **data-version marker**. It says which version of the underlying data the answer was built from (I use it in Part 3.5).
- The starting threshold is **0.85**, as the assignment says. I did not assume it was best; I test it in Part 3.

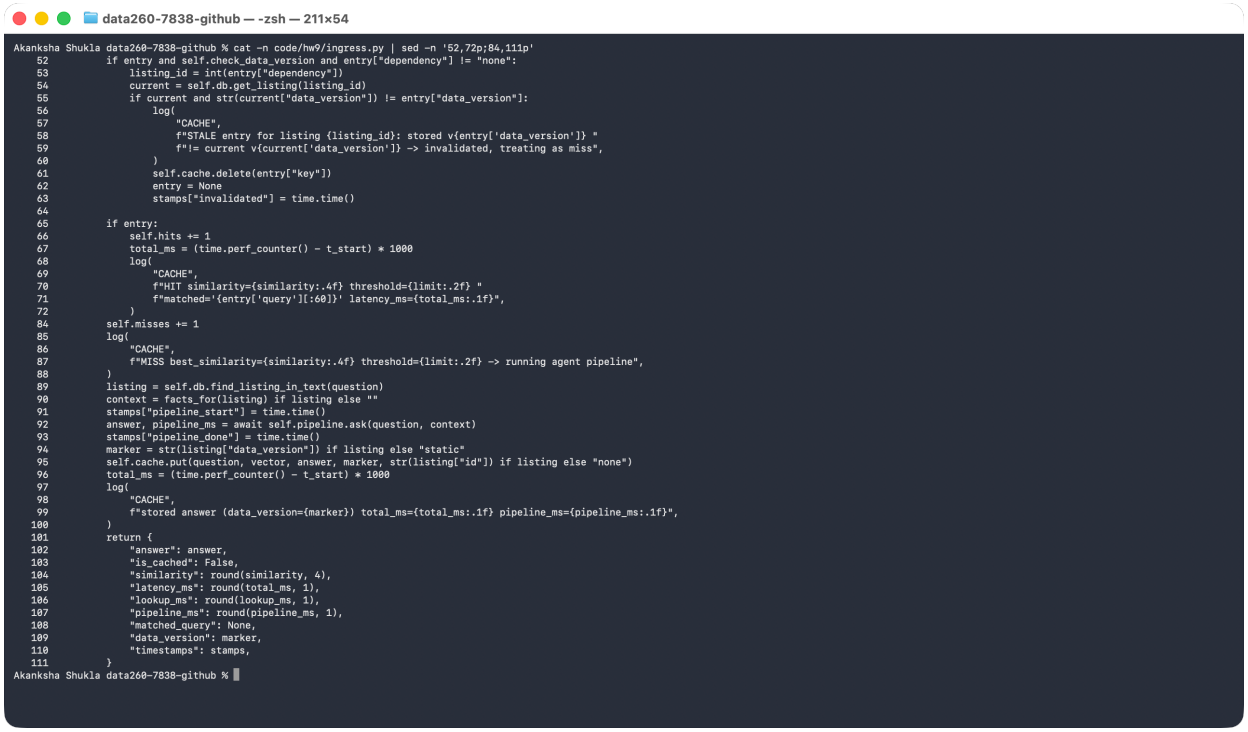
How I measure

- Each response carries **is_cached**, the **similarity** score and the **latency** in milliseconds.
- The server logs every hit (with its similarity) and every miss, and I record timestamps before and after the embedding, the lookup and the pipeline call.

Screenshot - my semantic cache code: Redis vector search, lookup and store



Screenshot - my ingress handler: lookup, hit/miss logging and timestamps



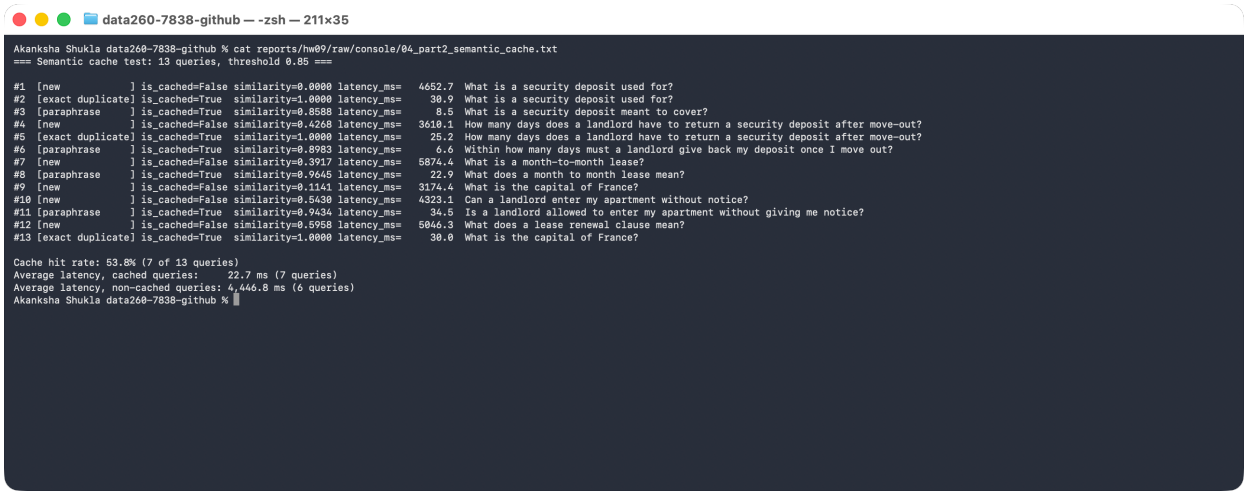
```
data260-7838-github — -zsh — 211×54

Akanksha Shukla data260-7838-github % cat -n code/hw9/ingress.py | sed -n '52,72p;84,111p'
    52          if entry and self.check_data_version and entry["dependency"] != "none":
    53              listing_id = int(entry["dependency"])
    54              current = self.db.get_listing(listing_id)
    55              if current and str(current["data_version"]) != entry["data_version"]:
    56                  log(
    57                      "CACHE",
    58                      f"STALE entry for listing {listing_id}: stored v{entry['data_version']} "
    59                      f"!= current v{current['data_version']} -> invalidated, treating as miss",
    60                  )
    61                  self.cache.delete(entry["key"])
    62                  entry = None
    63                  stamps["invalidated"] = time.time()
    64
    65          if entry:
    66              self.hits += 1
    67              total_ms = (time.perf_counter() - t_start) * 1000
    68              log(
    69                  "CACHE",
    70                  f"HIT similarity={similarity:.4f} threshold={limit:.2f} "
    71                  f"matched='{entry['query'][:60]}' latency_ms={total_ms:.1f}",
    72              )
    84          self.misses += 1
    85          log(
    86              "CACHE",
    87              f"MISS best_similarity={similarity:.4f} threshold={limit:.2f} -> running agent pipeline",
    88          )
    89          listing = self.db.find_listing_in_text(question)
    90          context = facts_for(listing) if listing else ""
    91          stamps["pipeline_start"] = time.time()
    92          answer, pipeline_ms = await self.pipeline.ask(question, context)
    93          stamps["pipeline_done"] = time.time()
    94          marker = str(listing["data_version"]) if listing else "static"
    95          self.cache.put(question, vector, answer, marker, str(listing["id"]) if listing else "none")
    96          total_ms = (time.perf_counter() - t_start) * 1000
    97          log(
    98              "CACHE",
    99              f"stored answer (data_version={marker}) total_ms={total_ms:.1f} pipeline_ms={pipeline_ms:.1f}",
   100          )
   101          return {
   102              "answer": answer,
   103              "is_cached": False,
   104              "similarity": round(similarity, 4),
   105              "latency_ms": round(total_ms, 1),
   106              "lookup_ms": round(lookup_ms, 1),
   107              "pipeline_ms": round(pipeline_ms, 1),
   108              "matched_query": None,
   109              "data_version": marker,
   110              "timestamps": stamps,
   111          }
Akanksha Shukla data260-7838-github %
```

My test: 13 queries

- I used 13 queries on purpose: new questions, exact duplicates and paraphrases. For example #1 is new, #2 repeats it exactly, and #3 rewords it.
- The exact duplicates scored **1.0000**. The paraphrases scored 0.8588, 0.8983, 0.9645, 0.9434, all above 0.85, so they were hits. New and unrelated questions (such as 'What is the capital of France?') scored low and were misses.

Screenshot - the 13 queries with is_cached, similarity and latency



```
data260-7838-github — -zsh — 211×35

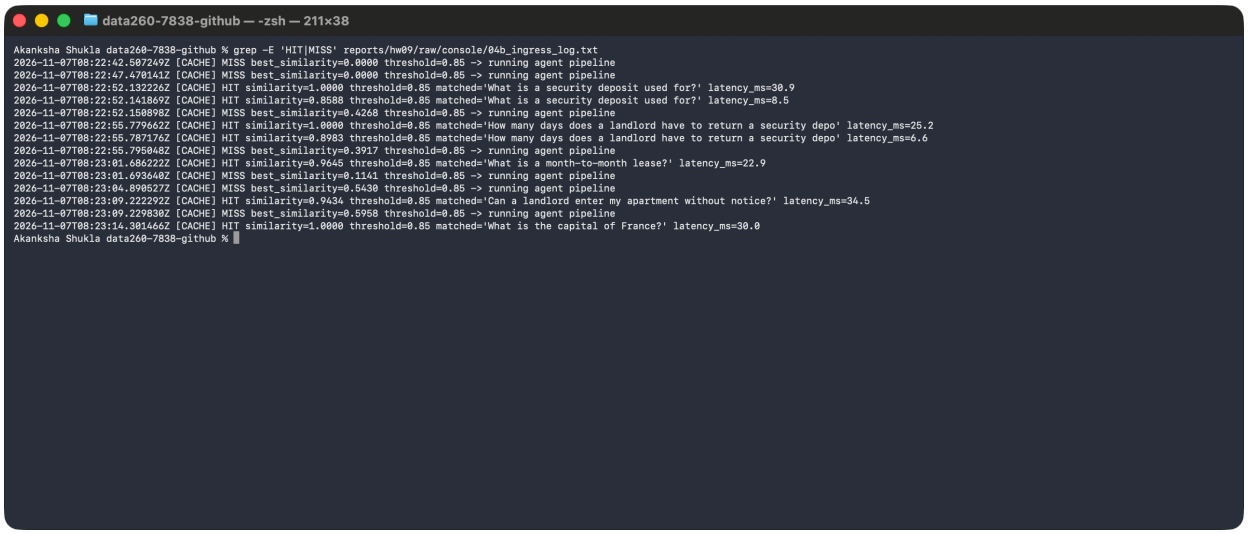
Akanksha Shukla data260-7838-github % cat reports/hw09/raw/console/04_part2_semantic_cache.txt
=== Semantic cache test: 13 queries, threshold 0.85 ===

#1  [new            ] is_cached=False similarity=0.0000 latency_ms=  4652.7  What is a security deposit used for?
#2  [exact duplicate] is_cached=True  similarity=1.0000 latency_ms=    30.9  What is a security deposit used for?
#3  [paraphrase     ] is_cached=True  similarity=0.8588 latency_ms=     8.5  What is a security deposit meant to cover?
#4  [new            ] is_cached=False similarity=0.4268 latency_ms=  3610.1  How many days does a landlord have to return a security deposit after move-out?
#5  [exact duplicate] is_cached=True  similarity=1.0000 latency_ms=    25.2  How many days does a landlord have to return a security deposit after move-out?
#6  [paraphrase     ] is_cached=True  similarity=0.8983 latency_ms=     6.6  Within how many days must a landlord give back my deposit once I move out?
#7  [new            ] is_cached=False similarity=0.3917 latency_ms=  5874.4  What is a month-to-month lease?
#8  [paraphrase     ] is_cached=True  similarity=0.9645 latency_ms=    22.9  What does a month to month lease mean?
#9  [new            ] is_cached=False similarity=0.1141 latency_ms=  3174.4  What is the capital of France?
#10 [new            ] is_cached=False similarity=0.5430 latency_ms=  4323.1  Can a landlord enter my apartment without notice?
#11 [paraphrase     ] is_cached=True  similarity=0.9434 latency_ms=    34.5  Is a landlord allowed to enter my apartment without giving me notice?
#12 [new            ] is_cached=False similarity=0.5958 latency_ms=  5046.3  What does a lease renewal clause mean?
#13 [exact duplicate] is_cached=True  similarity=1.0000 latency_ms=    30.0  What is the capital of France?

Cache hit rate: 53.8% (7 of 13 queries)
Average latency, cached queries:     22.7 ms (7 queries)
Average latency, non-cached queries: 4,446.8 ms (6 queries)
Akanksha Shukla data260-7838-github %
```

- The server log shows the same story: each hit lists its similarity, and each miss says it is running the agent pipeline.

Screenshot - server log lines for each hit and miss



What I measured

Measure	Result
Cache hit rate	53.8% (7 of 13 queries)
Average latency, cached queries	22.7 ms
Average latency, non-cached queries	4,446.8 ms

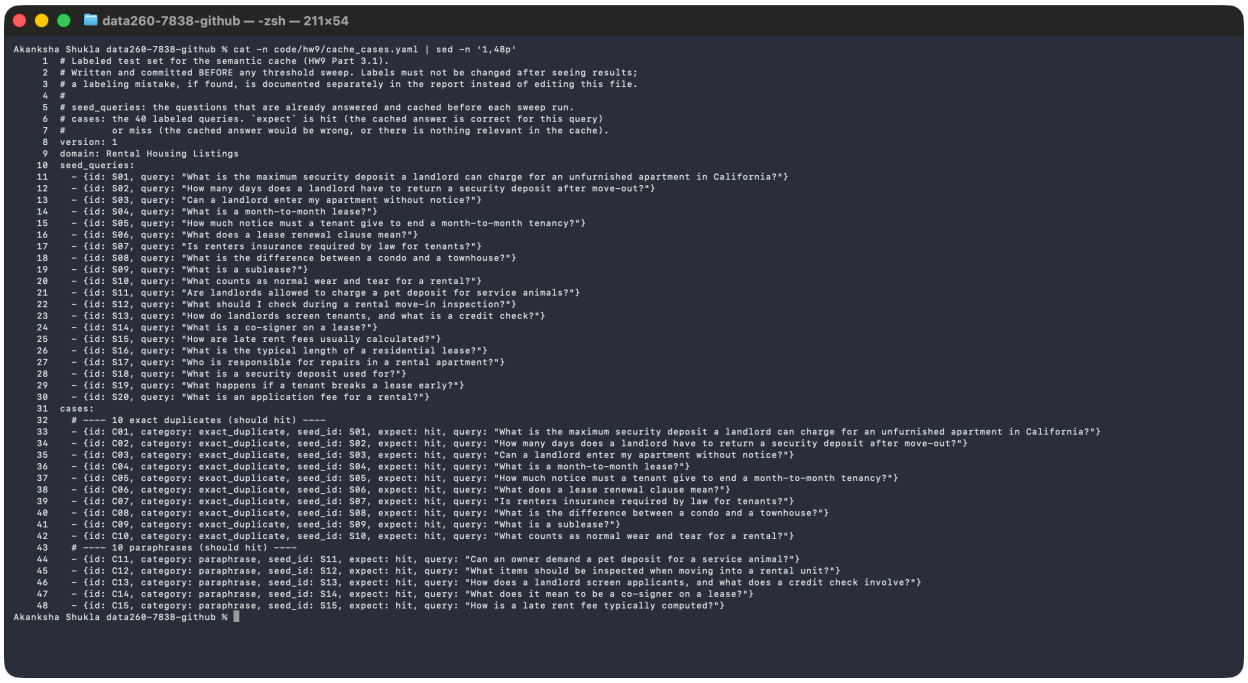
- These are the numbers I actually measured. I did not aim for a particular speed-up.
- The hit rate depends on my test mix. I put several repeats and paraphrases in the 13 queries, so it would be lower for real traffic with mostly new questions.
- Before the measured runs I send one throw-away request, so the embedding model and the language model are already loaded. In my first try the first query took 47,681 ms only because of that loading, which would have been unfair to count.

Part 3 - Threshold economics and stale answers

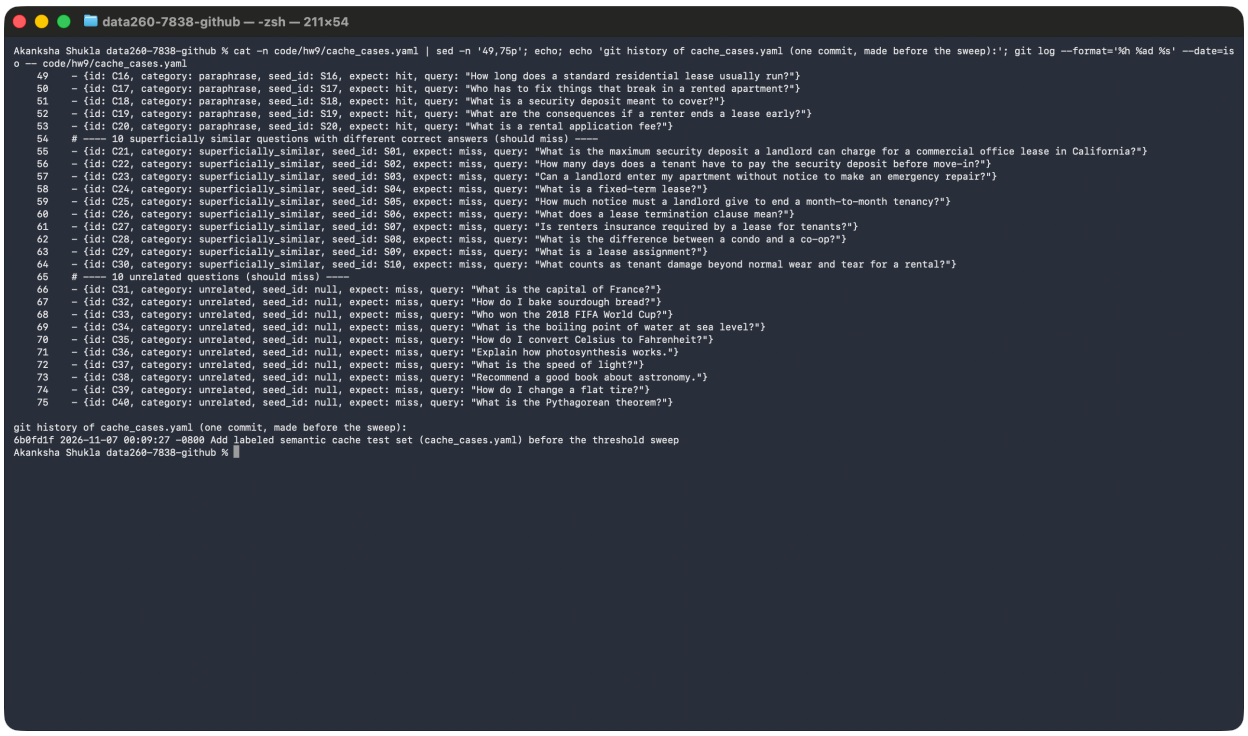
3.1 My labeled test set (cache_cases.yaml)

- To judge a threshold fairly I need questions where I **already know** whether the cache should hit or miss. So I wrote **40 labeled queries** before running anything.
- **10 exact duplicates** (should hit), **10 paraphrases** that mean the same thing (should hit), **10 superficially similar** questions that look alike but have a different correct answer (should miss), and **10 unrelated** questions (should miss).
- Example of a superficially similar pair: 'How much notice must a **tenant** give to end a month-to-month tenancy?' versus the same sentence with **landlord**. The words are nearly identical, but the answers differ.
- The file also lists 20 **seed queries**. These are the questions already cached before each test run. A hit counts as correct only if it matches the seed it is supposed to match.
- I **committed the file to git before I ran any sweep** (commit 6b0fd1f), and it has only that one commit. I did not relabel any case after seeing results.

Screenshot - cache_cases.yaml (part 1 of 2)



Screenshot - cache_cases.yaml (part 2 of 2) and its single commit in git



3.2 Threshold sweep

- I ran the 40 cases at five thresholds: 0.75, 0.80, 0.85, 0.90 and 0.95.
- For every threshold I started with a cache holding the 20 seed answers. A hit costs only the lookup time. A miss costs the lookup plus the real time my agent pipeline needed for that query (I measured each query once on the live system).
- **Hit rate** = hits divided by 40. **False-hit rate** = false hits divided by the 20 queries that should have missed. **Mean latency** = the average over all 40.

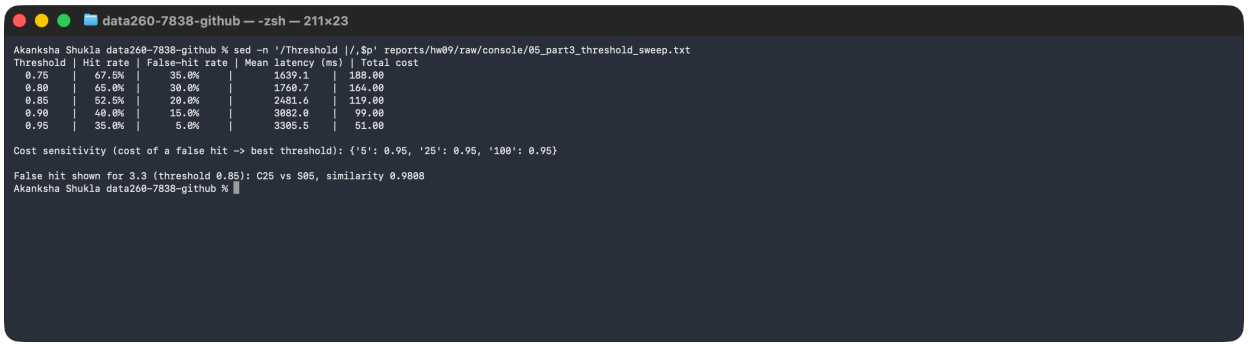
- The code below is the core of my sweep: how I decide hit, false hit and miss for each case, and how I compute the hit rate, false-hit rate, mean latency and total cost.

Screenshot - my sweep and cost-model code



Threshold	Hit rate	False-hit rate	Mean latency (ms)	Total cost
0.75	67.5%	35.0%	1,639.1	188
0.80	65.0%	30.0%	1,760.7	164
0.85	52.5%	20.0%	2,481.6	119
0.90	40.0%	15.0%	3,082.0	99
0.95	35.0%	5.0%	3,305.5	51

Screenshot - my threshold sweep output



What the table tells me

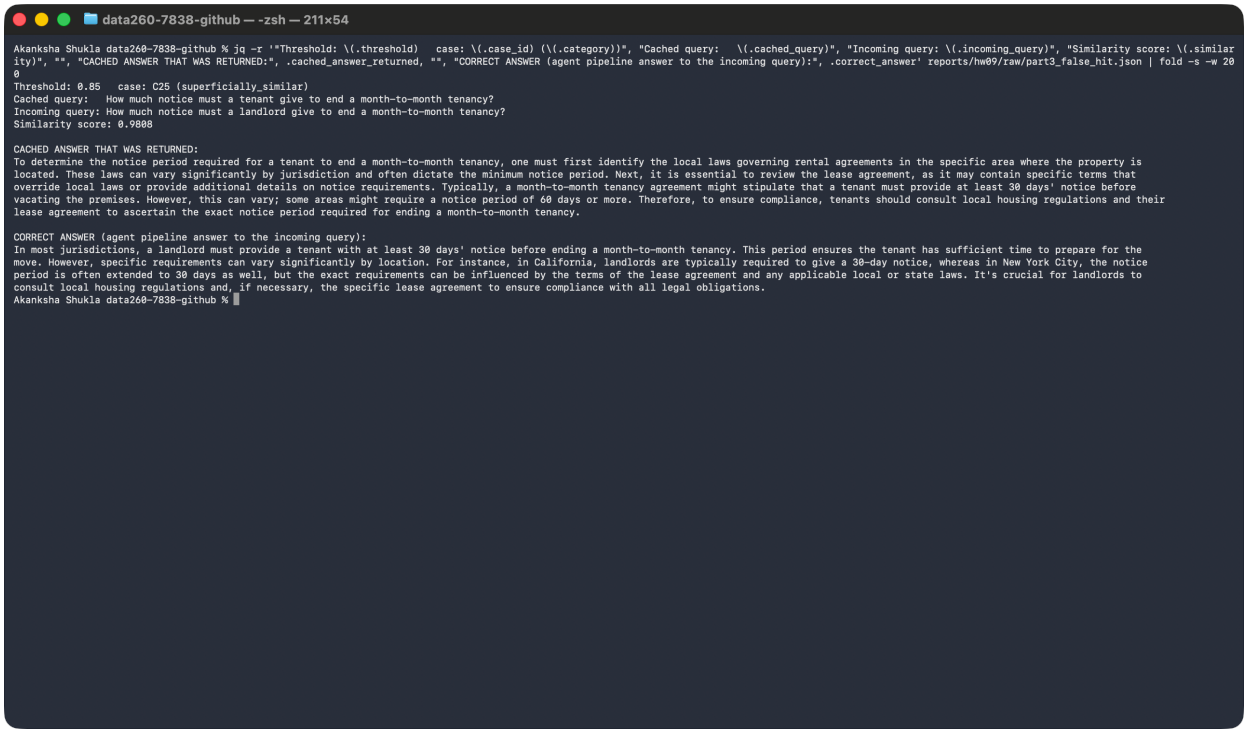
- When I **raise** the threshold, false hits drop (35% down to 5%), but the hit rate also drops (67.5% down to 35%). With fewer hits, more questions run the slow agents, so the mean latency goes up.

- At my starting threshold of 0.85 I got 4 false hits (scores 0.85 to 0.98) and also missed 3 paraphrases that should have hit (scores 0.80 to 0.83). So 0.85 was not ideal in either direction.

3.3 One false hit in full

- **The query already in the cache:** How much notice must a tenant give to end a month-to-month tenancy?
- **The incoming query:** How much notice must a landlord give to end a month-to-month tenancy?
- **Similarity score:** 0.9808 (threshold 0.85), so the cache returned the stored answer.
- The screenshot shows the cached answer that was returned and the correct answer, which is what my agent pipeline produced when I ran the incoming question for real.

Screenshot - the false hit: both queries, similarity, cached answer and correct answer



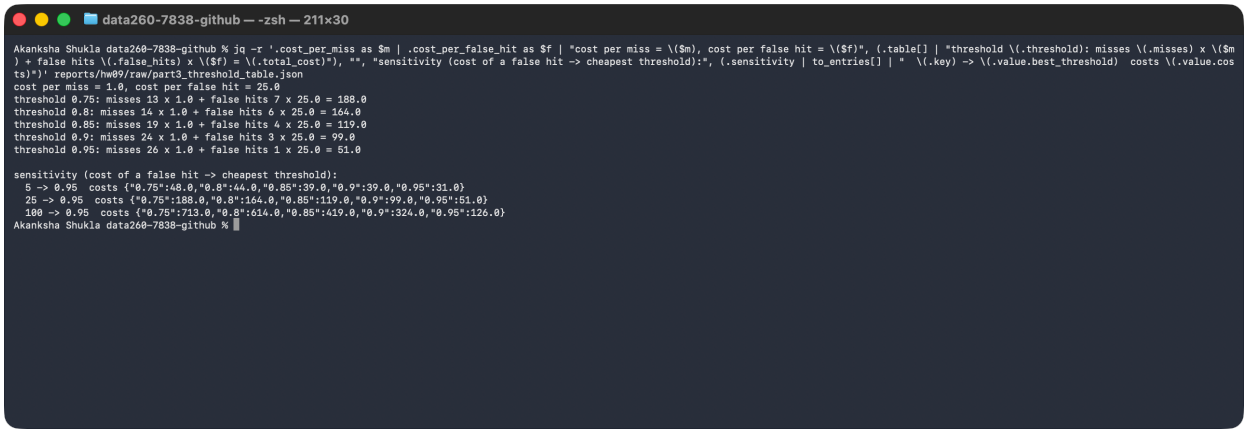
- **Why the embedding model was confused:** the two sentences are almost word for word the same, and only one word differs (tenant versus landlord). An embedding captures mostly the topic, which here is notice to end a month-to-month tenancy. It gives very little weight to who must give the notice, so the score came out at 0.98.
- **Why it matters:** the cached answer is about what a tenant has to do. The renter asking the second question needs to know what the landlord has to do. Giving the wrong one could mislead a real person.

3.4 My cost model and the threshold I would ship

- **Cache-miss cost = 1 unit.** A miss means one extra full run of my agents, about 3 to 8 seconds of local model time plus the user waiting. I use this as my base unit.
- **False-hit (support) cost = 25 units.** A wrong answer about a lease or a deposit can lead a renter to a bad decision and a support follow-up. I treat that as far more costly than one model run, so I picked 25 times a miss.
- **Total cost at a threshold** = (number of misses x 1) + (number of false hits x 25).

- I also tried 5 and 100 for the false-hit cost to see whether my choice depends on that guess.

Screenshot - cost at each threshold and the sensitivity check

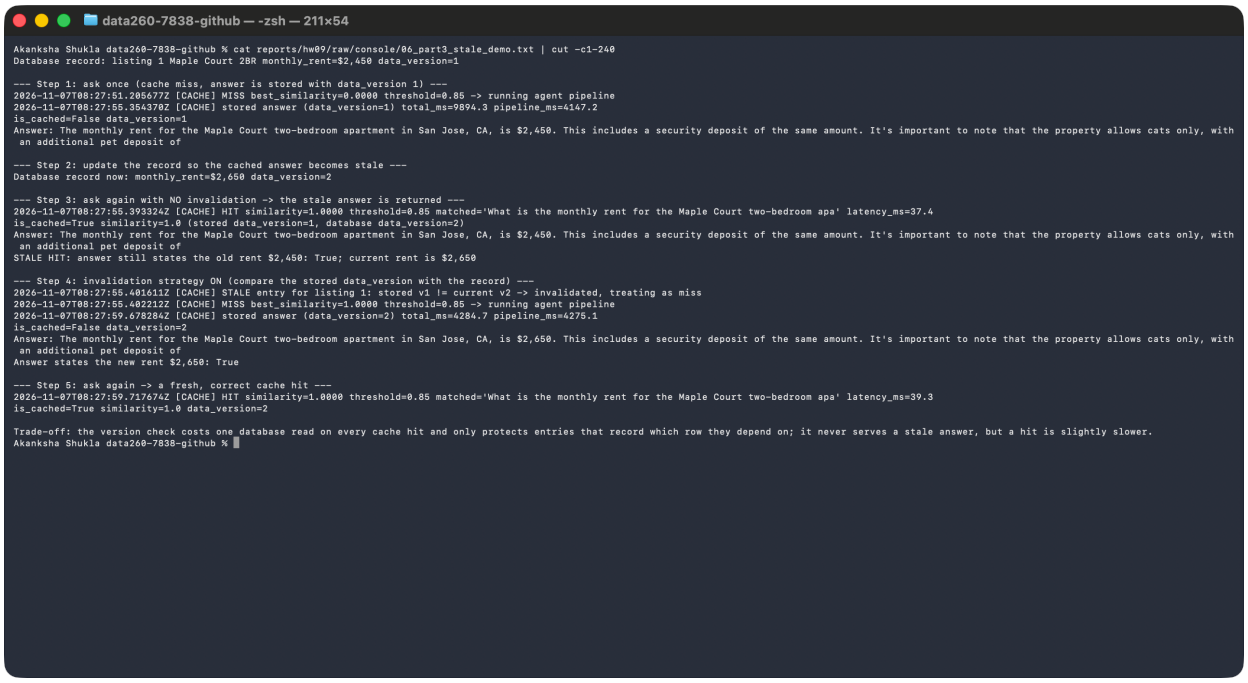


- **I would ship threshold 0.95.** It has the lowest total cost (51), and it stayed the cheapest when I changed the false-hit cost to 5 or to 100. So my decision does not depend on my exact guess.
- **What I give up:** the hit rate falls to 35.0%, because many correct paraphrases (scores 0.80 to 0.88) no longer count as hits. I accept more slow answers in return for fewer wrong ones.
- **A limit I found:** the tenant versus landlord pair (0.98) still gets through at every threshold, so no threshold can remove this kind of error. To fix it I would need a second check, for example comparing the key words or using a stronger model.

3.5 Stale answers and invalidation

- **1. Chosen query:** 'What is the monthly rent for the Maple Court two-bedroom apartment?' Its answer depends on one record in my s7838_rel database: the Maple Court listing, with rent $2,450.
- **2. I updated that record:** I changed the rent to $2,650. My update function also raises the row's data_version from 1 to 2.
- **3. Stale hit:** with no protection, I asked the same question again and the cache returned the old answer that still said $2,450.
- **4. My invalidation strategy:** every cache entry stores the data_version of the record it was built from. On a hit I compare it with the record's current data_version. If they differ, I delete the entry and answer the question again with the agents.
- **Result:** the demo logged 'STALE entry ... invalidated', the agents produced a new answer with $2,650, and the next time I asked I got a correct cache hit.

Screenshot - stale hit, then invalidation using the stored data-version marker



- **5. Trade-off of my strategy:** I pay one database read on every cache hit, so hits are a little slower. It also only protects answers that record which row they depend on, and it needs a version number that changes on every update. In return I never serve an answer that I know is out of date.

Part 4 - Self-improving multi-agent system

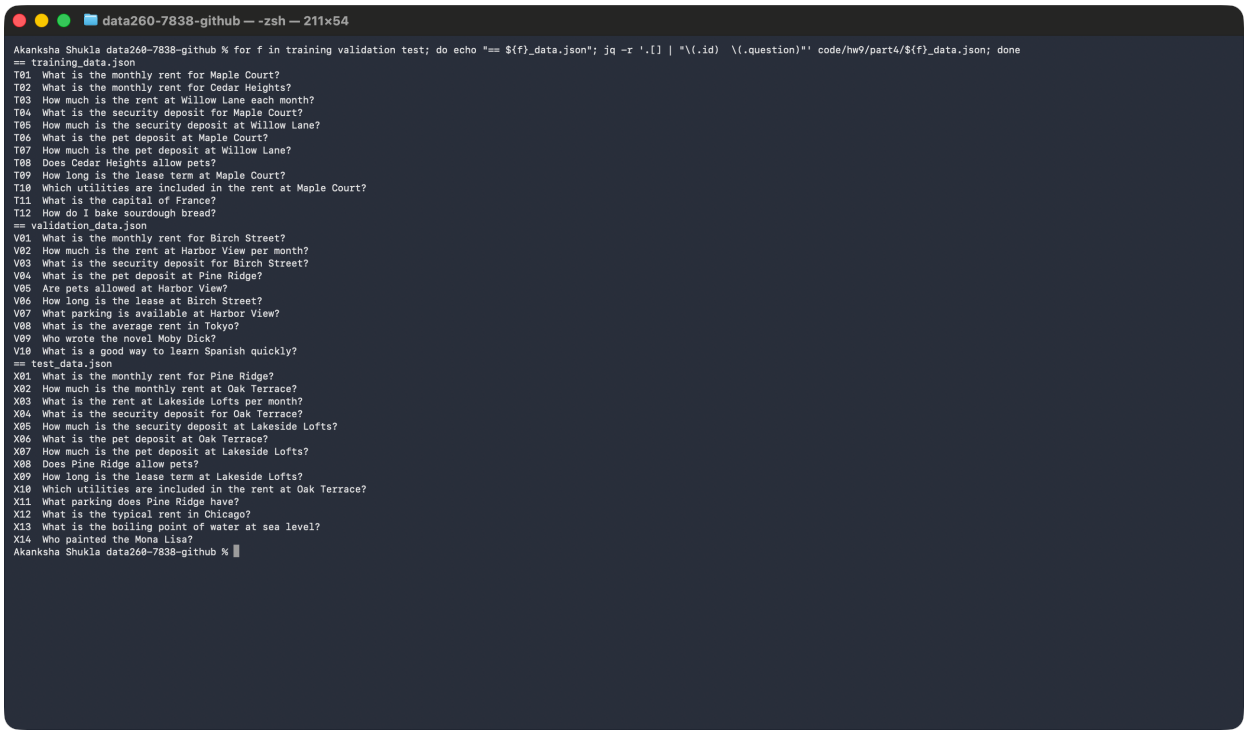
What I wanted to do

- I built two agents that work together. The **Answering Agent** writes an answer. The **Evaluator Agent** checks it against trusted facts and, if it is wrong, tells the Answering Agent what to fix.
- I also let the system **remember lessons** from repeated mistakes, and I only keep a lesson if it really helps on separate validation data.

The data I created

- **reference_data.json** (the ground truth): 8 rental properties, each with facts such as monthly rent, security deposit, pet policy, lease term, utilities and parking.
- **training_data.json** (12 questions), **validation_data.json** (10) and **test_data.json** (14). These files contain **only questions**. The correct answers live only in the reference dataset.
- Some questions are deliberately **unrelated** to my properties (for example 'What is the capital of France?'). For those, the correct behavior is to say 'I don't have that information'.
- I used the training questions while developing, the validation questions to test lessons, and the test questions **only** for the final evaluation.

Screenshot - my training, validation and test questions



Screenshot - lessons.json and the reference dataset

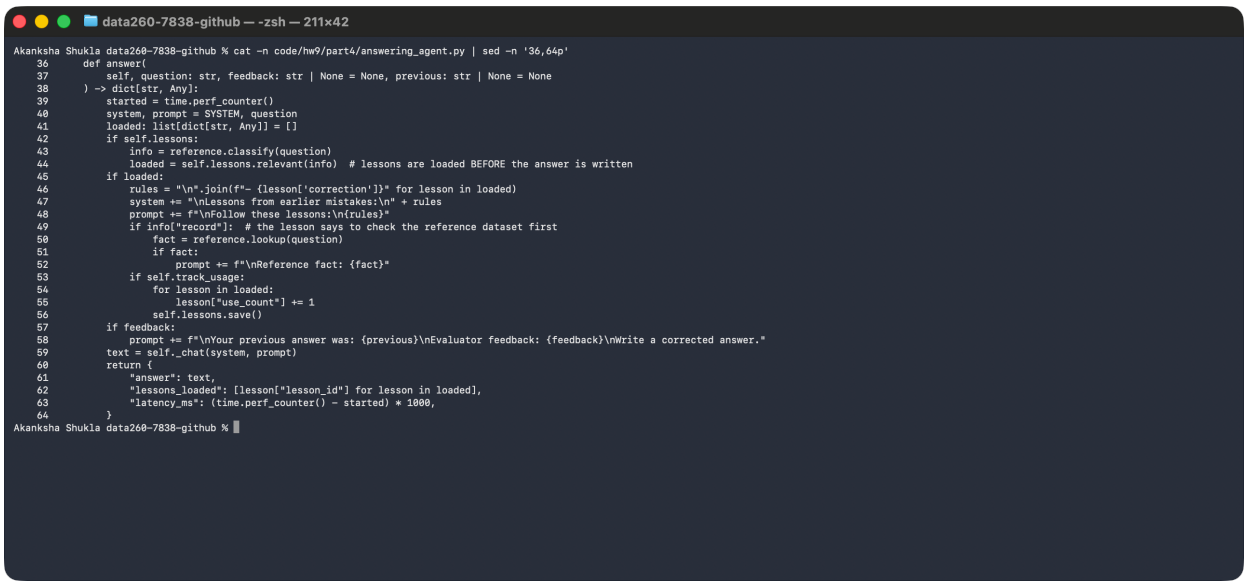


4.1 Repairing failed answers with evaluator feedback

- **Step 1:** the Answering Agent receives a question and writes an initial answer (qwen2:7b, temperature 0, seed 7838).
- **Step 2:** the Evaluator checks the answer against the reference dataset.
- **Step 3:** the Evaluator returns **PASS or REVISE**, a short explanation, and feedback when the result is REVISE.
- **Step 4:** on REVISE, the Answering Agent gets that feedback and writes a revised answer. **Step 5:** the Evaluator checks the revised answer. **Step 6:** the system returns the final answer and evaluation.
- **Stopping rule:** at most **two attempts**. After the second answer the system stops and reports the final result, even if it is still REVISE.

- **How the Evaluator decides:** it finds which property and which fact the question asks about, then checks that the right number or words appear in the answer. For unrelated questions it checks that the agent declined instead of guessing.

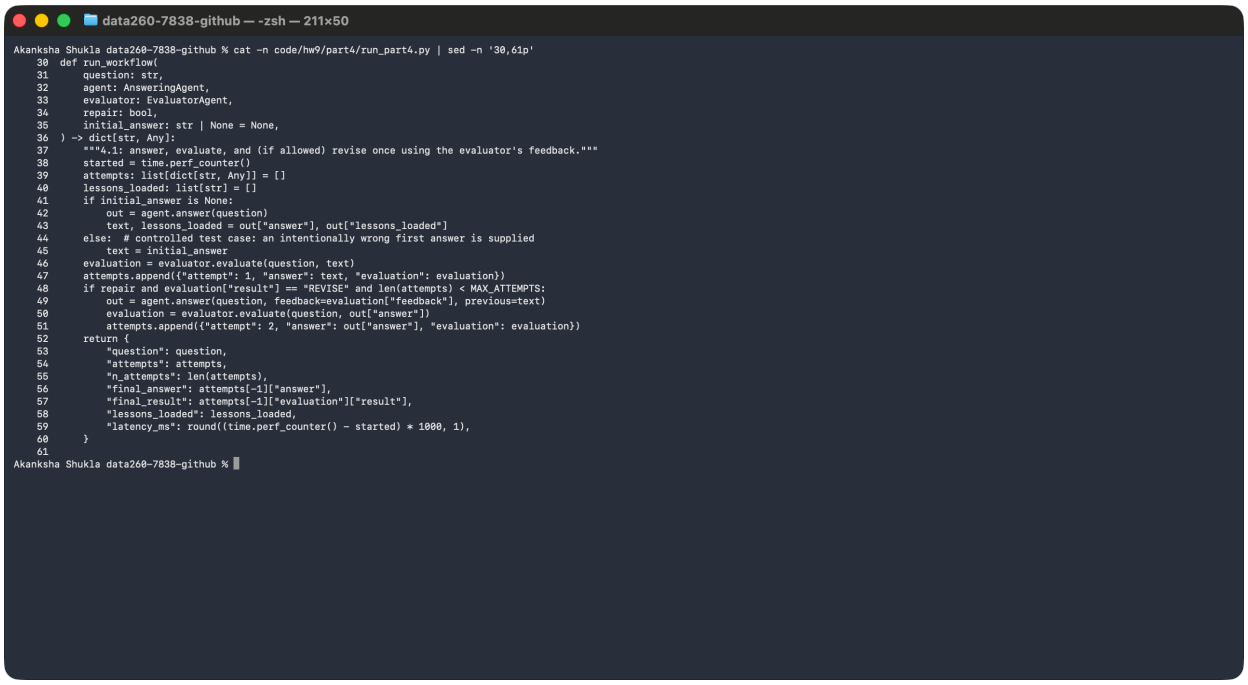
Screenshot - my Answering Agent



Screenshot - my Evaluator Agent



Screenshot - my revision workflow with the two-attempt limit



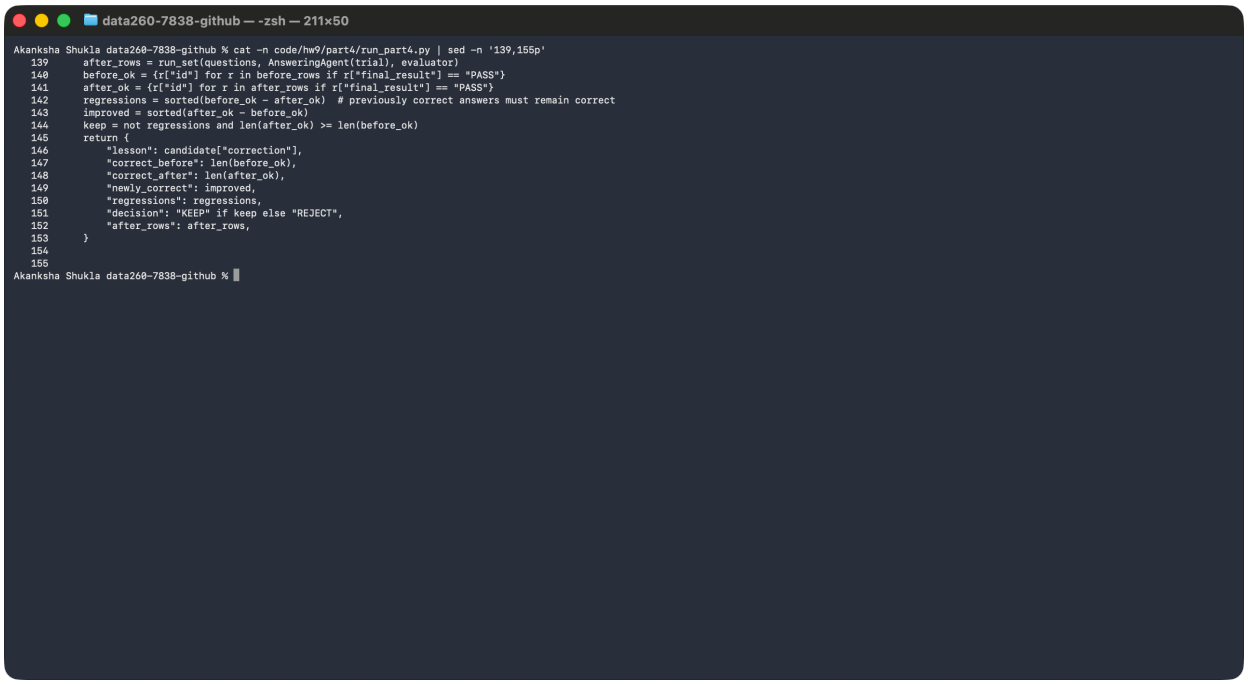
4.2 Saving and reusing lessons

- When the same failure pattern shows up at least twice in the training run, the system creates a lesson in **lessons.json** with lesson_id, failure_pattern, correction, created_at and use_count.
- Before the Answering Agent writes an answer, it **loads the lessons that match the question** and follows them. Each time a lesson is used, its use_count goes up by one.
- On my training questions the plain agent failed all 12: 10 times it gave a missing or wrong property value, and 2 times it answered an unrelated question from general knowledge. The first failure was T01 (Maple Court rent). Because the same pattern repeated, the Evaluator created lesson_001 from those failed answers and saved it. That produced **two lessons**: 'check the reference dataset before answering about a listed property' and 'say you don't have that information for unrelated questions'.

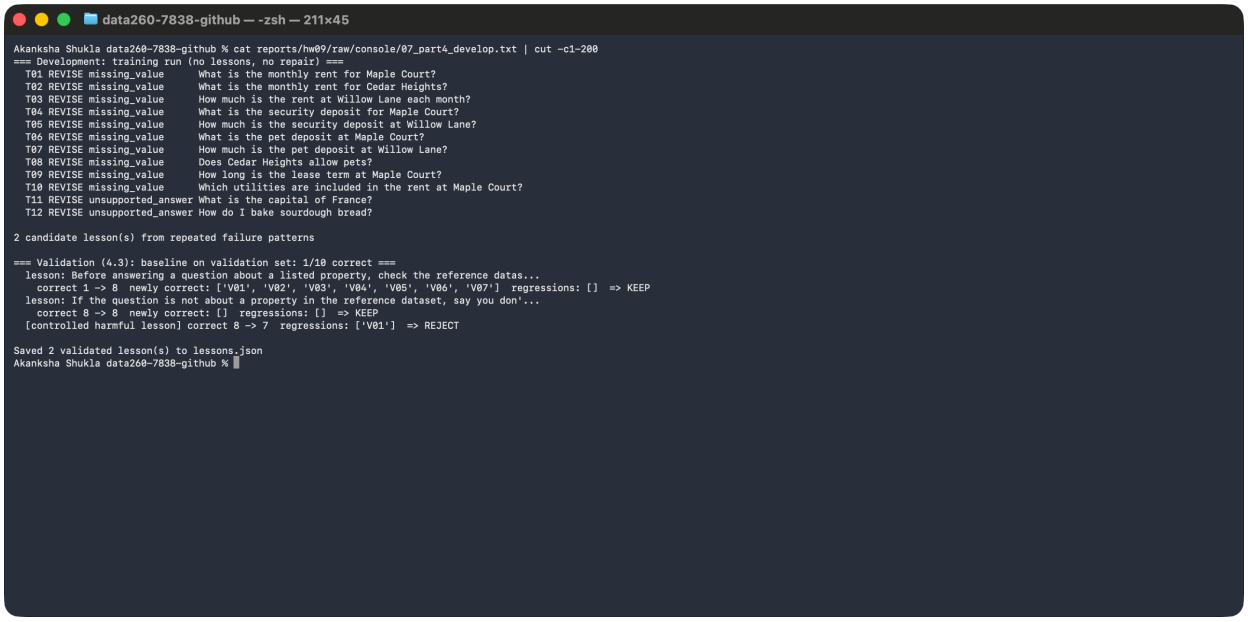
4.3 Validating a lesson before I keep it

- I run the validation set **before** and **after** adding a candidate lesson. I keep the lesson only if: previously correct answers stay correct, failures improve or stay the same, and unrelated questions do not start getting unsupported answers.
- The property lesson raised validation from **1 of 10 to 8 of 10** with no regressions, so I **kept** it.
- The unrelated-topics lesson changed nothing (8 to 8), so I kept it as 'unchanged'.
- **Controlled check:** I also tested a deliberately harmful lesson ('ignore the reference data and always say $3,000'). It broke a question that was correct before (V01), so the gate **rejected** it. That shows the gate really can catch a bad lesson.

Screenshot - my validation gate (compare before and after)



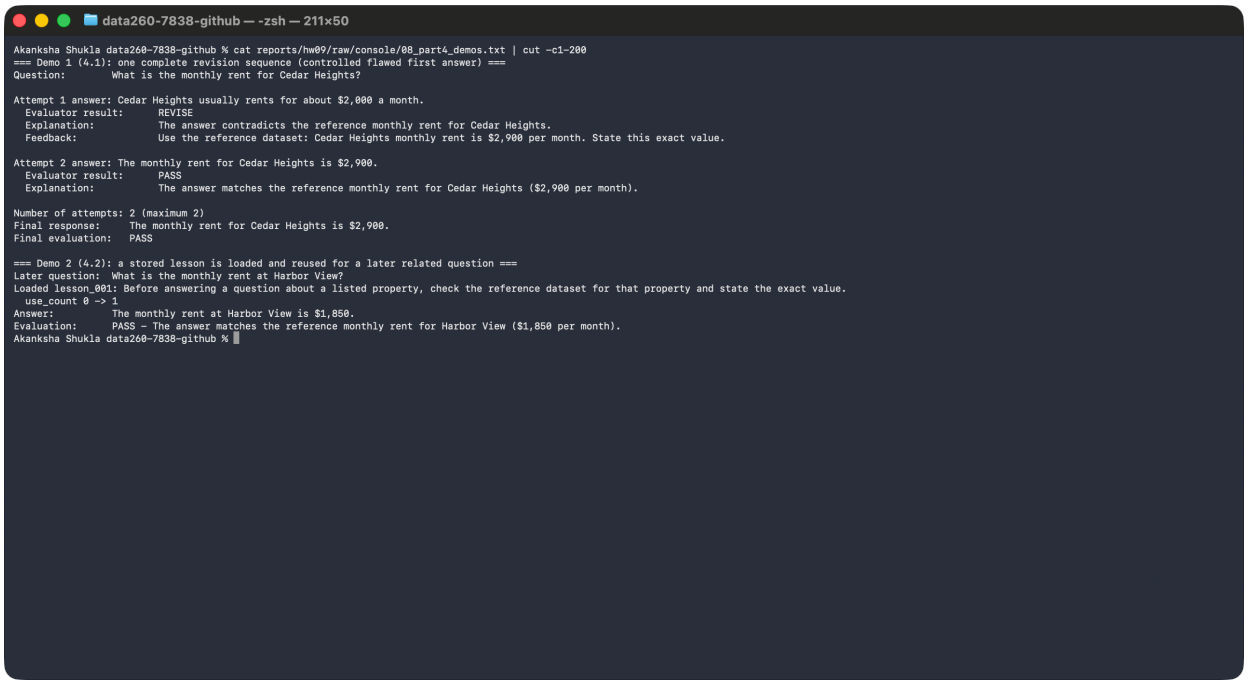
Screenshot - training run, candidate lessons and validation decisions



Required demonstrations

- **A complete revision sequence.** I used a controlled case where the first answer is **intentionally wrong**: 'Cedar Heights usually rents for about $2,000'. The Evaluator said **REVISE** and gave feedback with the correct value. The revised answer said $2,900, and the final evaluation was **PASS after 2 attempts**.
- **A lesson created from earlier failed answers, then reused later.** lesson_001 was created from the failed training answers above. For a later question, 'What is the monthly rent at Harbor View?', the system loaded **lesson_001** before answering (use_count went from 0 to 1) and gave the correct answer, $1,850.

Screenshot - the revision sequence and the lesson reuse

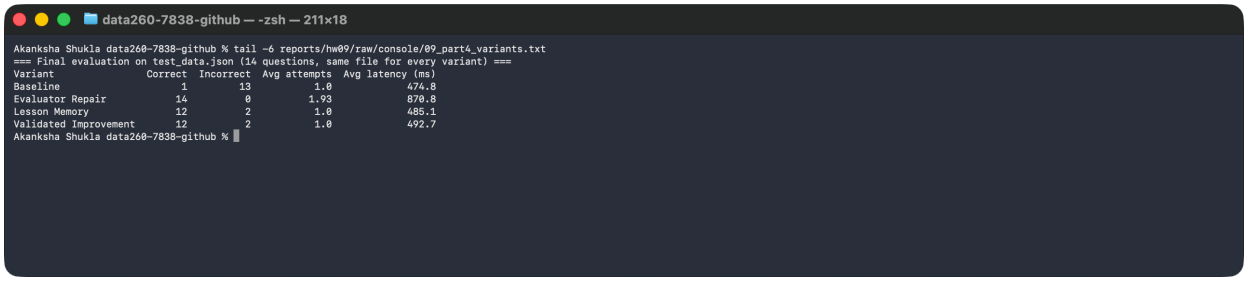


4.4 Four system variants on the test set

- **Baseline:** the Answering Agent alone, with no evaluator feedback and no lessons.
- **Evaluator Repair:** the agent revises failed answers using the Evaluator's feedback.
- **Lesson Memory:** the agent loads all the saved lessons.
- **Validated Improvement:** the agent loads only lessons that passed validation.
- I ran the same test_data.json (14 questions) on all four and did not change it between runs. The test file was never used to create lessons.

Variant	Correct answers	Incorrect answers	Average attempts	Average latency (ms)
Baseline	1	13	1	474.8
Evaluator Repair	14	0	1.93	870.8
Lesson Memory	12	2	1	485.1
Validated Improvement	12	2	1	492.7

Screenshot - my final test results for the four variants



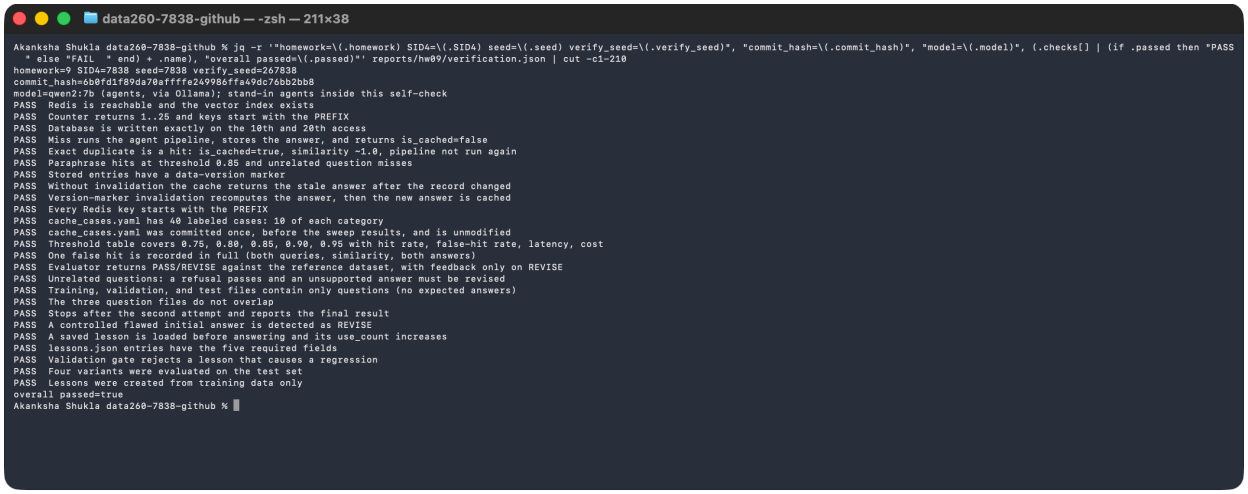
My explanation

- **Agent communication:** the Answering Agent sends its answer to the Evaluator. The Evaluator sends back PASS or REVISE with feedback. On REVISE, I put that feedback into the Answering Agent's next prompt.
- **Lesson reuse:** I match a saved lesson to a question by its kind (a listed property, or unrelated). The matched lessons are loaded before the answer is written, and the property lesson also makes the agent look up the reference fact first.
- **Stopping condition:** at most two attempts, then the system stops and reports the final evaluation.
- **Validation process:** each candidate lesson is tested on validation_data.json before and after it is added. I keep it only if nothing that was correct gets worse.
- **One limitation:** Evaluator Repair got 14 of 14 because the Evaluator's feedback contains the correct fact, and a real system would not always have such a fact. The lessons got 12 of 14: two general-knowledge questions (boiling point of water, who painted the Mona Lisa) were still answered, because the 7B model often ignores a 'say you do not know' lesson.
- **Other honest notes:** my test set is small (14 questions), so these numbers are a rough guide. Lesson Memory and Validated Improvement are equal because both of my lessons passed validation.

Self-check (verification.json)

- I wrote a script, make verify-hw09, that starts the pieces and runs a quick smoke test. A smoke test only checks that the basics work.
- It checks Redis and the vector index, the counter and the 10th/20th-view sync, a miss then a hit, paraphrase and unrelated behavior, stale answers and invalidation, that every Redis key starts with my PREFIX, the 40 labeled cases, the threshold table, the question files, the two-attempt stop, lesson loading and the validation gate.
- It uses stand-in agents so it does not depend on the language model, and it only creates temporary files. It never changes my application code.
- It writes verification.json with the homework number, SID4, commit hash, model and settings, SEED and VERIFY_SEED, each check, and pass or fail.
- Result: **25 of 25 checks passed.**

Screenshot - my verification result



AI use

1. What I used an AI assistant for, and what I did myself

I used Claude Code to make sure that my fixed configuration (SID4 7838, port 8638, prefix s7838, seeds), and my Docker, Redis, Kafka, and Ollama environment related configuration is perfect.

2. One AI output that was wrong or unsuitable

My first lesson-creation code made one lesson per field (rent, security deposit, pet deposit). With only one or two failures per field it left pet policy, lease term, utilities, and parking uncovered, so the test run would have shown little improvement. My first "controlled harmful lesson" for the validation gate was also too weak: it was not rejected (validation stayed at 5 of 10), so it proved nothing. In addition, the first Part 2 run reported 47,681 ms for the first query because it included loading the embedding model and the language model.

3. How I detected the problem or verified the result

I found these by reading the per-question validation results (the unrelated-topics lesson changed nothing and the harmful lesson caused no regression) and by comparing the 47,681 ms first query with the 3,500 to 6,400 ms of the other misses.

4. What I changed and why it works now

I grouped repeated failures by failure type and question scope, so one lesson covers all listed-property facts and one covers unrelated questions. I replaced the harmful lesson with one that tells the agent to ignore the reference data, and the validation gate now rejects it because it breaks a previously correct answer (V01). I added an unrecorded warm-up request before the measured runs. The validated lesson raised validation accuracy from 1 of 10 to 8 of 10 with no regressions.

Where everything is in my repository

- **code/hw9/:** all my HW9 code, cache_cases.yaml, and part4/ (the two agents, the datasets, the reference dataset and lessons.json).
- **reports/hw09/:** RUN_LOG.txt (real console output with timestamps), raw/ (JSON and CSV results), METRICS.md, AI_USE.md, verification.json, my screenshots and this report.